

Aveo Protocol v3

Aveo Protocol v3 - Compliance Desk + kits Solana

Especificação de MVP para Bruno + Codex e Caio + Cursor. Revisão de 30/09/2026, com cronograma antecipado para prontidão até 08/10, baseada na spec v1 de 29/09, no comparativo Aveo x MPL-3643 v3 e no pedido do Bruno de 30/09, 00h02.

Status: proposta para implementação, não software testado. Fontes públicas conferidas em 30/09; nenhum SDK MPL-3643 ou programa Aveo foi executado nesta revisão. IDs, layouts e dependências ainda não confirmados são gates explícitos. Este documento não autoriza publicar o repositório, usar dados reais, solicitar alpha em nome de Bruno, inscrever ou submeter o projeto.

0. Decisão para começar hoje

Construir **Aveo Compliance Desk**: uma prova de conceito operacional que responde:

Esta carteira pode mover este ativo agora? Qual prova sustenta a resposta? Se a prova vencer ou for revogada, quais ativos são afetados e como o operador fecha o incidente?

A infraestrutura genérica não é a novidade. A Metaplex já documenta SAS, múltiplos attestors, trust scope, onboarding e enforcement. A Aveo conserva a elegância da ideia original, mas muda a promessa: **a mesma prova sintética reutilizada em dois ativos, decisões distintas por emissor e um ciclo verificável de diagnóstico, incidente e correção.**

Começar pela **Rota B: SAS real + Token-2022 + hook próprio mínimo**, em devnet, com testes locais. Não depender de alpha para iniciar, não reproduzir os quatro programas Metaplex e não chamar isso de implementação do MPL-3643. A **Rota A: aplicação sobre MPL oficial** substitui o backend somente após acesso e testes. Não construir dois backends on-chain completos em paralelo.

Primeira demo

- Verificador A emite uma atestação SAS sintética para X.
- Alfa e Beta são dois ativos sintéticos com políticas independentes. Ambos aceitam essa mesma prova de A, sem nova emissão de KYC.
- Transferências para X passam inclusive fora da interface Aveo.
- Y apresenta prova de B. Alfa aceita B; Beta aceita apenas A. Alfa permite; Beta nega e explica.
- A prova X expira ou é fechada por signer autorizado. O Desk agrupa os dois ativos afetados. Na Rota B, entrada e saída de X falham mesmo com o Desk desligado.
- Uma nova prova válida é emitida. X assina a vinculação; o Desk verifica a correção e registra a transferência permitida.

Isso prova composição e fluxo operacional. Não prova unicidade de mercado, demanda institucional, identidade real ou conformidade jurídica.

1. Alterações da v1

V1	V2	Motivo
Multi-verificador como contribuição principal	Desk de incidentes e decisões explicáveis	MPL já cobre multi-attestor e trust scope.
Prova com escopo por mint	Prova de fatos reutilizável + policy por mint	Portabilidade não deve exigir reemitir prova por ativo.
Schema único presumido para duas credentials	Pares permitidos (credential, schema)	Schemas SAS pertencem a credentials.
Prova só do destinatário	Prova de remetente e destinatário no hook B	Mostrar perda de elegibilidade de holder existente.
UI secundária	Desk pequeno como produto	Fechar tarefa operacional, não apenas demonstrar primitivas.
Sem decisão sobre alpha	B imediata; A condicionada; C de redução	Começar sem prometer compatibilidade inexistente.

Preservar Token-2022, SAS real, falha fechada, testes fora da UI, dados sintéticos, lockfiles e divisão de

módulos. Antes de editar, inspecionar o repo atual: a ausência de código relatada em 29/09 não prova o estado de hoje.

2. Usuário e escopo

Usuário-alvo hipotético: operador de uma plataforma de emissão/administração de ativos permissionados. Tarefa: ver quem perdeu elegibilidade, entender os ativos afetados e fechar a pendência com evidência.

Hipótese comercial ainda não validada: operar vários ativos/verificadores precisa de uma visão comum de incidentes. Buscar reação de um operador antes de ampliar. Não inserir comprador, parceria, logo, TAM ou depoimento inventado.

Métrica de bancada: tempo entre alteração confirmada da prova, detecção pelo Desk e correção verificada; contar incidentes duplicados e divergências entre diagnóstico e transação. Não chamar isso de SLA.

Fora do MVP: KYC real, PII, ZK/Zcash/Zama, privacidade de saldos, bridge, credencial transferível, yield, recovery, forced transfer, DEX, token próprio, política regulatória completa, SaaS multiusuário, indexador de toda Solana, mainnet com ativos reais, auditoria e monitor 24/7.

3. Rotas e gates

Rota B - backend imediatamente codificável

aveo-sas-hook: programa próprio lê SAS real e autoriza transferências Token-2022; Desk diagnostica, prepara e registra operações. Sem Token ACL, thaw ou re-freeze. Bloqueio por hook, não por conta Frozen. Mints B são próprios; não prometer migração automática para MPL.

Até 01/10: confirmar SDK/IDs/layouts SAS e demonstrar resolução de extras e leitura real no hook. Se falhar, reportar bloqueio; não substituir SAS por JSON ou assinatura local e manter o pitch.

Rota A - somente após acesso e teste

mpl3643: SDK/programas oficiais recebidos no alpha. A documentação descreve mainnet early access, experimental/pré-auditoria; isso não confirma um SDK público ou devnet disponível à equipe.

Gate A: confirmar acesso, licença, versões, IDs, rede permitida de teste, grant de issuer e fixtures sintéticas; executar onboarding, transferência e incidente real. Solicitação a terceiros depende da decisão de Bruno. Se só houver mainnet, não usar ativos/dinheiro reais nem avançar silenciosamente: permanecer em B e registrar limitação.

O bootstrap MPL usa Token-2022 Frozen, Token ACL, Gate e compliance. Seguir SDK: init de compliance transfere mint authority irreversivelmente, então Gate/trust scope e configs que exigem autoridade original vêm antes. Não aplicar init Aveo ao mint MPL.

Adapter A deve executar prepare → transfer → finalize quando aplicável; mostrar thaw, transferência e estado final. Incidente: executar re-freeze oficial e confirmar Frozen por leitura. A documentação atribui ao cranker a responsabilidade de re-freeze; demonstrar a consequência de ele não rodar. Não afirmar que todo mint MPL revalida toda regra em cada transferência: hook é opcional/configurável.

Rota C - reduzir ou parar

Se não houver SAS real/enforcement fora da UI, entregar protótipo parcial claramente rotulado e reavaliar submissão. Até 03/10, se só houver infraestrutura genérica e nenhuma tarefa operacional demonstrável, reconsiderar Aveo. Não deslocar a equipe Passaporte sem acordo.

4. Stack

Camada	Escolha da contingência
Programa	Rust + Anchor 0.32.1; CLI, anchor-lang e anchor-spl alinhados.
Solana	CLI/Agave 2.3.0 da v1; confirmar compatibilidade no primeiro build.
Token	Token-2022 + Transfer Hook; dois mints sintéticos; sem PermanentDelegate/Confidential Transfer.
Prova	SAS real: credential, schema e attestation. JSON só em teste unitário do Desk.

Cliente	Node 22 + TypeScript; SDKs compatíveis fixados em lockfile, sem latest.
UI	React + TypeScript; três vistas pequenas.
Incidentes	SQLite local para demo, nunca fonte de autorização on-chain.
Rede	Validador local e devnet; confirmar implantação/binário SAS oficial e permitido.

Não inventar program ID SAS ou versão de SDK. Preencher config/deployments.devnet.json e docs/sas-compatibility.md com IDs, proveniência, codecs, commit e versões realmente usados. Teste local carrega SAS oficial, não um programa imitador. Alteração de toolchain exige lockfile/documentação atualizados.

5. Modelo de domínio

- **Proof:** fato afirmado sobre carteira por attestor, não autorização universal.
- **Policy:** condições de um emissor para um mint.
- **Eligibility:** resultado atual sob policy e Clock on-chain.
- **Binding:** ponteiro da carteira para a prova apresentada naquele mint; não cache de aprovação nem nova prova.
- **Incident:** problema operacional confirmado; não conclusão jurídica.
- **Evidence:** contas/slot/logs/resultado, não auditoria ou certificado de KYC.

Alfa aceita A ou B e exige KYC sintético. Beta aceita apenas A e exige KYC + accreditation sintéticos. X apresenta a mesma conta SAS A a ambos; Y/B só passa em Alfa. Políticas permanecem independentes.

6. Contrato on-chain aveo-v2

Congelar em docs/interface-v2.md antes de paralelizar. Contas/instruções abaixo são escolhas Aveo propostas, não APIs SAS/MPL oficiais.

6.1 IssuerPolicy

PDA proposto ["policy", mint] sob Aveo. Campos: mint, issuer_authority, sas_program_id, proof_domain [u8;32], até dois pares (credential, schema), require_kyc, require_accredited, active e policy_version u64.

Init exige assinatura da mint authority atual e define essa autoridade como issuer_authority; update exige o issuer_authority armazenado. Não aceitar init por qualquer caller ou apenas comparar uma chave autoindicada: isso permitiria tomar a policy de um mint alheio. Validar mint Token-2022, hook correto e ausência das extensões excluídas. SAS program ID é fixado pela implantação verificada, não escolhido livremente pelo caller; policy não pode apontar a um programa imitador. Schema pertence à credential. Criar uma credential e schema para A e outra para B com mesmo layout semântico; Alfa aceita ambos, Beta só A. Não presumir schema global compartilhado entre credentials.

Sem superadmin global. Depois do bootstrap, remover hook update authority se aplicável e verificar; se permanecer, explicitar que uma alteração pode contornar enforcement. Policy inativa bloqueia entrada/saída; isso é pausa demo, não resgate. Mudança pode afetar holders existentes; documentar poderes do issuer e upgrade authority.

6.2 Payload SAS aveo-eligibility-v2

```
version: u8 = 2
subject_wallet: bytes[32]
proof_domain: bytes[32]
kyc_pass: bool
accredited_pass: bool
```

Traduzir para tipos/codec SAS reais; bytes podem usar VecU8 com tamanho validado em 32. Validade vem de Attestation.expiry, sem campo duplicado conflitante. Domínio cobre a família demo, não um mint específico.

SAS fornece credential/schema/signer/nonce/expiry/tokenAccount. Nonce/PDA não provam sujeito sozinhos. Confirmar a semântica/tokenAccount permitido em prova não tokenizada; não atrelar ao ATA de um único ativo e chamar de portabilidade.

Só signer SAS autorizado emite/fecha. Revogação demo: close da atestação por signer autorizado e rejeição subsequente. Se o fluxo não funcionar na implementação real, registrar falha do gate.

6.3 EligibilityBinding

PDA proposto ["binding", mint, subject_wallet]. Campos de layout fixo: versão, mint, subject_wallet, credential, schema, attestation. Um por mint/carteira; dois bindings podem apontar à mesma prova.

set_binding exige assinatura da carteira sujeito e valida contas SAS, par, payload/sujeito e elegibilidade no registro. O hook relê contas SAS vivas em cada transferência; não usar eligible=true armazenado no binding. Renovação/rebind exige de novo assinatura da carteira. Usar nonce novo para a prova renovada e documentar a derivação oficial; não depender de recriar silenciosamente uma conta fechada no mesmo endereço. Servidor não assina por investidor; runner controla apenas carteiras sintéticas.

6.4 ExtraAccountMetaList

Execute usa contas padrão Token-2022 e PDA oficial ["extra-account-metas", mint]. Extras: policy; binding/proof/credential/schema do owner de source e de destination; demais contas exigidas pela interface real.

Remetente = owner do source token account, não transfer authority/delegate. Destinatário = owner do destination token account. Derivar bindings destes owners/mint, não de campos da UI.

Gate: resolver binding via dados das token accounts; resolver endereços SAS dos campos do binding por mecanismos oficiais de account-data metas/seeds. Validar a cadeia de dependência em TS/Rust. Gerar offsets do layout real fixo, documentar discriminador/tamanho e testar fixture compartilhada. Não adivinhar offsets.

Se inviável, propor estratégia/layout menor e testar de novo fora da UI. Nunca deixar caller escolher prova arbitrária nem trocar enforcement por API. Não colocar sujeitos em lista global de extras por mint.

6.5 Validação do hook

1. Confirmar contexto real de Transfer Hook, flag transferring e interface oficial; recusar chamada direta fora do contexto.
2. Validar source/destination Token-2022, mint/policy correta e ativa.
3. Validar binding PDA/owner/layout/mint/sujeito para ambas as partes.
4. Validar owners/PDAs/layouts SAS pelo código oficial; credential/schema/attestation coincidem com binding; schema.credential coincide com attestation.credential.
5. Exigir par permitido, layout Aveo v2, domínio correto, signer atualmente autorizado. Schema pausado nega por escolha conservadora Aveo, não regra universal atribuída ao SAS.
6. Sujeito do payload = owner correspondente; fatos exigidos positivos.
7. expiry > Clock.unix_timestamp. Demo não aceita expiry=0 ou prova sem prazo. Conta ausente/fechada/malformada nega.
8. Ambas passam: sucesso. Falha: erro estável, nenhum saldo alterado na transação fracassada.

Hook apenas lê. Limitar tamanho do payload e aceitar somente o layout exato pinado; trailing bytes, versões ou lengths desconhecidos negam. Sem RPC/HTTP, CPI de emissão, auto-renovação ou alteração de policy. Sem dependência de SQLite/monitor/servidor. Medir CU, tamanho de transação e resolução de contas antes de prometer viabilidade.

6.6 Mint/burn e poderes

Mint/burn não são apresentados como cobertos pelo hook. Bootstrap atesta a tesouraria com prova A válida e ambos os fatos positivos, cria bindings para os dois mints e emite saldo sintético só a ela; remover mint authority depois, se aplicável, e confirmar por leitura. Documentar burn por holder e poderes residuais. Não alegar cobertura universal.

B usa estado padrão de conta Initialized, não Frozen, e não usa freeze authority para incidentes: mint sem essa autoridade ou autoridade de bootstrap removida com readback. Sem PermanentDelegate. Upgrade/hook config authority visíveis no README; não chamar demo de imutável.

7. Desk e adapter

Contrato em packages/contracts/, independente de SDK experimental:

```
type BackendId = 'aveo-sas-hook' | 'mpl3643';
type Verdict = 'eligible' | 'ineligible' | 'unknown';
type Capability = 'transfer' | 'renew-binding' | 'refreeze';
interface EligibilityBackend {
  capabilities(): readonly Capability[];
  inspect(input: InspectInput): Promise<EligibilitySnapshot>;
  planTransfer(input: TransferInput): Promise<UnsignedPlan>;
  verifyOutcome(input: VerifyInput): Promise<OperationEvidence>;
}
```

Definir auxiliares antes de uso: `InspectInput` = cluster/mint/wallet; `snapshot` = backend, slot/commitment, horário observado, `policy/version`, `verdict`, razões, `source accounts`, `expiry` e ações. `UnsignedPlan` = instruções/contas e resumo legível dos efeitos, sem `private key`. `Evidence` = `signature` se existir, `confirmed/failed/unknown`, `logs/simulação` e `readback` pós-operação.

`Renew-binding/refreeze` são ações específicas de backend expostas por interfaces separadas ou métodos tipados adicionais; `capabilities` não bastam sem implementação. `unsupported` retorna erro explícito.

`RPC/layout/leitura inconsistente` = `unknown`; não mostrar aprovação nem preparar envio automático. Diagnóstico não garante transação futura: atualizar, simular e obter assinatura. `On-chain` decide no instante de execução.

Registrar contexto de leitura e lacunas; não juntar slots/RPCs conflitantes como `snapshot` atômico. Resultado desconhecido não autoriza `retry` cego: consultar assinatura e estado primeiro. Hash local não é prova auditada.

B expõe `transfer` + `renew-binding`, nunca `refreeze`. A só habilita operações oficialmente testadas. Esqueleto MPL mostra `not-integrated` e não contém IDs/mints fictícios nem respostas de sucesso `mock`.

8. Incidentes e ações

Indexar apenas os dois mints e carteiras/bindings demo. Guardar (`mint`, `wallet`, `proof`) e histórico local. Uma prova afetada agrupa ativos; `refreshcar/reiniciar` não duplica incidente.

Tipos: `PROOF_EXPIRED`, `PROOF_CLOSED`, `PROVIDER_REMOVED`, `POLICY_CHANGED`, `READ_UNVERIFIABLE`. Estados: `open` → `action-prepared` → `awaiting-confirmation` → `resolved`. Falha/`timeout` mantém aberto. Resolução exige leitura nova e evidência da operação/estado esperado.

Monitor por botão ou `scan:once`; não `watchdog` 24/7. Em local, usar avanço de relógio em harness suportado; `devnet` usa validade curta e espera. Sem loops públicos rápidos. Sem `scan`, painel fica desatualizado; em B o `hook` continua bloqueando prova inválida.

B: `attestor` emite prova nova, carteira assina `rebind`; `issuer` pode alterar somente sua `policy` em cenário explícito. Sem renovação secreta, `override` que aceite todos ou `attestor` global novo. A: `re-freeze` oficial testado; confirmar `Frozen`. Não confundir `freeze` com recolhimento/restituição.

9. UI mínima

1. Carteiras/ativos: `mint`, `wallet`, `attestor/schema`, `policy_version`, `expiry`, `verdict` e motivo; mostrar a prova compartilhada. Etiqueta fixa: “Provas sintéticas; demo `devnet`; sem KYC real”.
2. Transferência: origem/destino/amount, diagnóstico das partes, simulação, assinatura e resultado confirmado com `readback`. Sem `auto-send` em `refresh`.
3. Incidentes: causa, ativos afetados, slot/horário, ação possível e evidência antes/depois. Distinguir `hook B` de `re-freeze A`.

`Unknown` nunca verde. Motivos: “Beta não aceita B”, “A prova venceu”, “Prova de outra carteira”, “Leitura não confirmada”. UI explica, não concede permissão.

10. Erros estáveis

Documentar Rust → testes TS → UI e lado `source/destination`:

`NotTransferContext`; `PolicyMissingOrInactive`; `MintMismatch`; `BindingMissing`; `BindingSubjectMismatch`; `ProviderPairNotAllowed`; `SchemaMismatchOrPaused`; `AttestationMissingOrClosed`; `InvalidSasOwnerOrData`; `UnauthorizedAttestationSigner`; `SubjectMismatch`; `ProofDomainMismatch`; `RequiredFactMissing`; `AttestationExpired`; `MissingExtraAccounts`.

Desk: `BackendNotIntegrated`; `UnsupportedCapability`; `ReadUnverifiable`; `SimulationFailed`; `ConfirmationUnknown`. RPC falho não equivale a revogação.

11. Repo e divisão

Repositório próprio do Caio, a confirmar; não usar `contursi-labs`. Inspeccionar árvore atual, preservar trabalho e criar privado até decisão de publicação.

<code>programs/aveo-hook/</code>	# Bruno: <code>policy</code> , <code>binding</code> , <code>Execute</code> e extra metas
<code>packages/contracts/</code>	# ambos: interface v2 e erros
<code>packages/sas-client/</code>	# Caio: <code>bootstrap/emissão/close/codecs</code>

```

packages/backend-aveo/ # Caio: inspect/plan/verify reais
packages/backend-mpl/ # esqueleto not-integrated até gate A
app/ # Caio: Desk
scripts/ # bootstrap, scan:once, outside-ui-transfer
tests/onchain/ # Bruno: enforcement/adversarial
tests/client/ # Caio: incidentes, adapter, resolver
docs/ # interface-v2, threat-model, demo, provenance
config/ # manifests sem secrets

```

Ambos fecham interface e fixture SAS antes de paralelizar. Bruno entrega IDL/layout/seeds; Caio entrega provas reais/decoder compatível. Shared files só em PR acordado; não compartilhar branch/sessão de edição. Chaves/.env/seed phrases fora do Git/logs. Revisão humana de build/test/diff. A/B são verificadores simulados, não instituições reais.

12. Backlog de início

P0 - antes de UI

- ☐ Repo/commit base e v1 arquivada; interface-v2/threat-model.
- ☐ SDK/program IDs/layouts/expiry/close SAS confirmados; lockfile e deployment manifest.
- ☐ Credentials A/B, schemas próprios e atestação real de X emitida/lida/fechada.
- ☐ Spike extras: source/dest owners → bindings → contas SAS; transferência fora da UI.
- ☐ Mesma prova A validada em dois mints/policies.
- ☐ Expiração/close rejeitados; saldos preservados; CU/tamanho registrados.

Aceite P0 = teste on-chain real, não UI verde. Se falhar, registrar bloqueio antes de ampliar.

P1 - produto demonstrável

- ☐ Inspect/plan/verify com simulação/readback.
- ☐ Scan agrupado/deduplicado e persistência após restart.
- ☐ UI mostra prova reutilizada, razões e políticas divergentes.
- ☐ Renovação/rebind com assinatura da carteira e incidente corrigido.
- ☐ Script externo repete bloqueio com Desk desligado.
- ☐ README clone limpo, vídeo e poderes/limitações.

P2 - se houver tempo e gate A fechado

Adapter oficial mínimo para a mesma história. Sem yield/recovery/DEX ou reescrita geral. Se alpha exigir arquitetura diferente, discutir corte antes de implementar.

13. Matriz de aceite

Local automatizado; repetir happy path, revogação e transferência externa em devnet.

ID	Cenário	Resultado
T01	X: mesma conta SAS A, Alfa/Beta	Dois bindings, uma prova; ambos passam sem reemissão.
T02	Y/B	Alfa passa, Beta nega par não aceito.
T03	A com accreditation falsa	Alfa passa; Beta nega requisito.
T04	Binding/prova ausente ou KYC falso	Falha, saldos iguais.
T05	Prova X para Y; mint/domínio errado	Falha por validação de owner/PDAs/policy.
T06	Prova X expira, Desk desligado	Entrada/saída X falham em ambos; Desk só detecta no scan.
T07	Close SAS autorizado	Próximas transferências falham, readback comprova close.
T08	A removido só de Alfa	Alfa bloqueia; Beta segue válido enquanto prova não vencer.
T09	Renovar + X assina rebind	Prova nova validada; incidente resolve por readback/teste.
T10	Conta falsa/layout/schema/signer inválido	Falha fechada.
		Com extras: mesmas regras; sem

T11	Fora do app, com/sem extras	extras: falha, não bypass.
T12	Delegate envia saldo X	Hook usa owner X, não delegate.
T13	Update issuer/binding sem assinatura	Falha; chamada direta de Execute também falha.
T14	RPC/confirm timeout/leitura inconsistente	Unknown, incidente não fecha, retry não é cego.
T15	Scans repetidos/restart	Sem duplicação; reconcile com cadeia.
T16	Owner igual nas duas contas/amounts de fronteira	Duplicatas resolvidas; sem desvio de sujeito.
T17	Policy inativa/schema pausado/expiry zero ou igual Clock	Falha pela semântica v2.

MPL separado/opcional: conta nasce Frozen; thaw elegível; transfer SDK; perda de elegibilidade; cranker parado; re-freeze/readback. Sem acesso = NOT_RUN, não PASS. Testes B não comprovam MPL.

14. Segurança e privacidade

Só carteiras sintéticas; nenhuma carteira pessoal Bruno/Caio/terceiros como sujeito. Não inserir CPF, país real, documentos, scores ou hashes de KYC. Mesmo sem nome, carteira vinculável pode ser dado pessoal. Portátil não significa privada: prova/correlação são públicas.

SAS prova uma afirmação do attestor, não a verdade de KYC. Se A mentir, todos que confiam em A são afetados. Remoção de signer vivo invalida provas antigas pela escolha conservadora v2; explicar/testar, não vender como único modelo de revogação correto.

Clock on-chain rege validade. Mudança entre diagnose/simulate/send pode causar recusa correta. Mostrar poderes de issuer, upgrade/hook authority, mint e burn. Bloqueio não recolhe tokens nem assegura enforcement jurídico. Sem auditoria externa.

15. Marcos - prontidão antes da viagem

Plano revisado em 30/09. Datas-mira em São Paulo, não compromissos externos. A janela de 09 a 12/10 fica reservada a emergência/remoto, sem assumir disponibilidade do Bruno. Meta: demo pronta, clone limpo, revisão humana concluída e material de inscrição/submissão preparado até 08/10. Preparar não é enviar.

Data	Saída e gate
30/09	Contrato v2; emissão/leitura SAS; spike extras e esqueleto Desk.
01/10, quinta	Gate P0: hook lê SAS real; resolução de extras, close e compute demonstrados. Congelar backend: B por padrão; A só se acesso/rede/fluxo mínimo oficial já forem comprovados e não causarem reescrita.
03/10, sábado	Dois mints, mesma prova reutilizada, policies divergentes e transferência externa permitida/negada. Decisão go/no-go: sem esse caminho real, reduzir ou parar, não transferir a integração para a viagem.
04/10, domingo	Ciclo incidente → renovação/rebind → correção integrado; scan agrupado e UI mínima. Sem função nova ou segundo backend.
05/10, segunda	End-to-end devnet, inclusive close/expiração e script externo; principais testes de segurança e readback. Falhas restantes registradas como bloqueios, não escondidas por mocks.
06/10, terça	Feature freeze; matriz de aceite, README/lockfiles, threat-model, poderes e proveniência completos. Só correção de bug no core após isso; copilot isolado é stretch pré-planejado conforme seção 21, sem atrasar revisão.
07/10, quarta	Clone limpo e demo gravada; evidências e material de inscrição/submissão preparados sem enviar. Primeiro passe de revisão humana de código, privacidade, pitch e dependências/licenças. Corrigir os achados, repetir testes afetados, concluir

08/10, quinta

revisão humana e congelar release candidate. Bruno decide se autoriza inscrição/publicação/submissão antes da viagem ou em outro momento. Sem aprovação, material permanece pronto e não enviado. Somente buffer de emergência/remoto: correção crítica, regravação indispensável ou envio já autorizado. Nenhuma feature nova durante a viagem, integração MPL tardia ou expectativa de trabalho diário do Bruno.

09 a 12/10

Compressão: end-to-end antecipado de 07 para 05/10; feature freeze de 09 para 06/10; gravação de 10 para 07/10; revisão final de 11 para 08/10. Custo: cortar adapter MPL tardio e expansão P2; copilot só entra como stretch isolado se os gates verdes permitirem, sem sacrificar revisão. Reduzir polimento de UI e operar apenas dois mints/carteiras de fixture. Preservar SAS real, enforcement externo, readback, testes adversariais e revisão humana. Se P0 atrasar, o custo é redução de escopo ou não submissão de uma demo incompleta, não eliminar esses controles.

Não há estimativa de esforço nem evidência de código executado suficientes para garantir essas datas. O gate 03/10 deve confirmar a viabilidade do end-to-end em 05/10; se extras/SAS ainda estiverem bloqueados, a prontidão completa até 08/10 fica ameaçada. A revisão humana depende de tempo reservado por Bruno e Caio; não presumir que esteja agendado.

Regras consultadas: inscrição/submissão até 12/10/2026, 23h59 PT, equivalente a 13/10, 03h59 em São Paulo. O prazo oficial e os gates de autorização não mudam. Não trabalhar até esse limite nem presumir aprovação por proximidade do prazo. Verificar requisitos de cada participante; spec não promete elegibilidade ou prêmio.

16. Demo e pitch

Demo de aproximadamente três minutos: policies Alfa/Beta; prova X compartilhada; transferências externas; Y/B passa em Alfa e falha em Beta; close X; scan agrupa incidente; script externo falha com Desk parado; renovação/rebind; transferência permitida e resolução com evidência. Em A, substituir hook B pelo fluxo oficial/re-freeze efetivamente testado. Não filmar mock como integração real.

Pitch em português:

“Começamos pela dor da elegibilidade portátil para ativos em Solana. Durante a pesquisa conhecemos o MPL-3643 da Metaplex, que já cobre identidade, políticas e tokens permissionados. Não reivindicamos esses componentes como invenção. A Aveo é um Compliance Desk: uma prova sintética pode servir a dois ativos, cada emissor mantém sua política, e o operador acompanha a causa de uma perda de elegibilidade, os ativos afetados e a correção comprovada. Nesta demo, [backend realmente testado] aplica as regras on-chain, inclusive fora da nossa interface.”

B acrescenta: “Não integramos MPL-3643 nesta entrega. Usamos SAS e Token-2022; adapter oficial é próximo passo condicionado a acesso e testes.”

A história “ideia antes da descoberta” exige documentos/commits próprios datados; não significa prioridade global ou precedência sobre o desenvolvimento Metaplex. Não chamar MPL de só anunciado: a documentação diz mainnet early access.

Pitch em inglês:

“We started with portable eligibility for assets on Solana. During our research, we found Metaplex’s MPL-3643, which already covers identity, issuer policies and permissioned tokens. We are not claiming to have invented those building blocks. Aveo is a small compliance desk: one synthetic attestation can support two assets, each issuer keeps its own policy, and an operator can trace an eligibility incident through to a verified fix. Our demo uses [the backend actually tested] and shows on-chain enforcement outside our UI.”

B: “This submission does not integrate MPL-3643. Our proof of concept uses SAS and Token-2022; an official MPL adapter is future work, subject to access and testing.”

17. Submissão sem inflar novidade

Entregar README reproduzível, commit/lockfiles, arquitetura, PASS/FAIL/NOT_RUN, contas/mints/programs, transações confirmadas, evidência de recusas, vídeo e limitações. Transação rejeitada na simulação pode não ter signature confirmada: guardar logs/comando reproduzível, não inventar link explorer.

Já existe

Contribuição Aveo proposta

SAS, credentials/schemas e close
Token-2022/hook
MPL multi-attestor/trust scope/lifecycle
Produtos de compliance/monitoramento

Integração recortada para a demo.
Policy/binding/validação mínima própria na B.
Desk e adapter oficial somente se testado.
Hipótese de UX específica ainda por validar competitivamente.

Colosseum julga funcionalidade, impacto, novidade, UX, open-source/composição e plano de negócio. Código útil e composição contam; demo genérica de primitivos tem novidade fraca. O pivô não elimina esse risco nem garante prêmio. Publicação é decisão separada; licenças e artefatos alpha podem restringir redistribuição.

18. Prompts iniciais de implementação

Bruno / Codex

Ler spec/repo/v1 arquivada. Implementar só B/P0. Confirmar SAS real e resolver de extras primeiro; testar transferência externa antes de ampliar. Não inventar IDs/offsets nem usar mock no aceite. Fechar interface com Caio; entregar policy/binding/hook e testes adversariais. Reportar diff, comandos, versões e falhas. Não deploy mainnet, publicação ou submissão.

Caio / Cursor

Ler contrato acordado. Criar SAS client, credentials/schemas A/B e fixtures X/Y/tesouraria. Implementar adapter B, inspect/plan/verify e scan:once; UI depois do caminho real. MPL explicitamente not-integrated até gate A. Não editar contrato/programa sem PR combinado, guardar secrets ou vender unit/mock como end-to-end.

Cada tarefa deve devolver arquivos alterados, comportamento, testes/resultados, bloqueios, comandos reproduzíveis e diferenças da spec. Build verde não é aceite funcional.

19. Fontes verificadas nesta revisão

Documentação lida, não execução Aveo:

- MPL arquitetura/SAS/trust scope/SDK alpha: <https://www.metaplex.com/docs/smart-contracts/mpl-3643>
- Bootstrap/envelope/cranker: <https://www.metaplex.com/docs/smart-contracts/mpl-3643/getting-started>
- SAS contas/validade: <https://attest.solana.com/docs/attestations>
- SAS signers: <https://attest.solana.com/docs/credentials>
- SAS schemas/tipos: <https://attest.solana.com/docs/schemas>
- SAS PDA: <https://attest.solana.com/docs/helpers>
- SAS close: <https://attest.solana.com/docs/instructions/close-attestation>
- Hook: <https://solana.com/docs/tokens/extensions/transfer-hook>
- Extra metas/cliente: <https://solana.com/docs/tokens/extensions/transfer-hook-integration>
- Toolchain: <https://www.anchor-lang.com/docs/updates/release-notes/0-32-1>
- Regras/prazo/critérios:
<https://colosseum.com/legal/Crypto%20World%27s%20Fair%20Hackathon%20Rules.pdf>

V3 substitui a v2 como proposta de escopo; o contrato on-chain permanece aveo-v2. A inclusão dos kits não altera código, permissões ou decisões automaticamente.

20. Solana AI Kit - ferramenta de desenvolvimento, não runtime

Incluir no setup de 30/09 para Bruno/Codex e Caio/Cursor: <https://github.com/solanabr/solana-ai-kit>. O projeto fornece configuração/contexto, skills, regras e integrações para desenvolvimento Solana. Não é SDK de enforcement, não roda no hook e não garante que a IA deixe de inventar APIs.

Setup revisado, sem execução remota cega

1. Registrar commit do kit e submódulos escolhidos. Ler README e install.sh desse commit, licença e arquivos alterados antes de executar. O README contém links antigos solana-claude-config; usar o repo solicitado acima e validar o commit, sem copiar endereço antigo às cegas.
2. Instalar inicialmente em checkout temporário separado para comparar o diff. O instalador documenta --agents, que direciona conteúdo a .agents/ para ferramentas não-Claude. Com script local revisado: `bash install.sh --agents /caminho/do/checkout-de-teste`. Não executar `curl | bash` sem revisão.
3. Preservar configuração existente de Codex/Cursor, regras do projeto, .gitignore, .gitmodules, MCPs e

comandos. O installer pode sobrescrever diretórios e alterar ignore/config. Copiar/integrar somente o recorte aprovado; não rodar comandos de cleanup/deploy/update automaticamente.

4. Selecionar contexto Solana core, Anchor, Token-2022, testes e segurança. Configuração deve referenciar esta spec, interface-v2 e docs pinadas. Verificar no cliente real que Codex e Cursor carregam esse conteúdo; colocar arquivos em .agents/ não comprova descoberta automática. Adaptar o arquivo de instruções suportado pela versão de cada editor conforme necessário.
5. MCP é opcional, não gate P0. Não habilitar todos por padrão, usar credenciais em arquivos publicados nem permitir deploy/mainnet/sends por efeito de uma configuração importada. Revisar licenças de submódulos separadamente: licença do kit não torna todos os conteúdos redistribuíveis.
6. Registrar docs/dev-tooling.md: commit, conteúdos selecionados, diff, regras de cada editor e teste de carregamento. Não instalar simultaneamente plugin e full kit sem verificar duplicação.

Aceite D01: ambos os editores conseguem localizar a spec/interface e explicar as seeds/contas acordadas, citando os arquivos. D02: diff de setup preserva regras existentes e não contém secrets. D03: geração proposta usa versões/APIs verificadas e passa revisão/build/test; resposta convincente de IA não substitui isso. Se o setup tomar tempo demais, disponibilizar contexto revisado manualmente e seguir P0; tooling não pode atrasar o hook real.

21. Solana Agent Kit - copilot de leitura, stretch goal

Incluir no plano como integração opcional de produto: <https://github.com/sendai/solana-agent-kit>. A documentação V2 apresenta arquitetura de plugins e integração com frameworks de IA; a Aveo propõe um plugin próprio de leitura. Não há plugin Aveo/SAS/MPL comprovado nesta revisão. Não é necessário para cumprir o MVP.

O copilot responde a perguntas como “Por que X foi recusada em Beta?” ou “Quais ativos esta prova vencida afeta?”, usando snapshots/evidências já produzidos pelo Desk. A diferença operacional é uma explicação acessível e rastreável; IA em compliance não deve ser vendida como novidade inédita por si só.

Prazo e teto de esforço

Somente depois do feature freeze de 06/10, com P0/P1 e testes principais verdes. Spike isolado no máximo de meio dia em 07/10; precisa estar integrado e testado a tempo da revisão final de 08/10. Se não cumprir gates, desligar/remover da release e filmar o Desk determinístico. Sem extensão do prazo ou trabalho essencial na viagem.

O freeze de 06/10 congela core, schemas, policy, hook, adapter e workflow de transação. O stretch é uma exceção isolada previamente planejada, atrás de feature flag `COPILOT_ENABLED=false` por padrão, sem alterar o core. Não modificar a release candidate revisada após 08/10 para encaixar IA.

Arquitetura e fronteira de segurança

Desk determinístico → snapshot/evidência sanitizados → plugin Aveo somente-leitura
→ Solana Agent Kit / modelo → explicação com fontes

- Enforcement continua exclusivamente no programa determinístico. O modelo não define eligibility, policy, binding, autorizações, ações de renovação ou estado de incidente. Não altera eligible/ineligible/unknown vindo do Desk.
- Nenhuma chave, seed phrase, wallet conectada, signing provider ou permissão de transação na rota do copilot. Não copiar o quick start com KeypairWallet/private key. V2 aceitar wallet provider sem private key não o torna read-only: um provider pode assinar.
- Criar plugin com somente três ações propostas: `getEligibilitySnapshot`, `getIncidentEvidence`, `listAffectedDemoAssets`. São contratos Aveo a implementar, não exports oficiais. Não carregar plugins token/DeFi/NFT/blinks nem exportar todo `agent.actions` de uma instância com ações de escrita.
- Se o construtor da versão pinada exigir wallet interface, confirmar no código a viabilidade de implementação sem poder de assinar: nenhum signer conectado; todos os métodos de assinatura/envio lançam erro, public key é apenas identificador sintético quando exigido. Testar isso. Não inventar um parâmetro oficial `readOnly=true` e não fornecer dummy secret para satisfazer construtor. Se a integração não funcionar sem signer, gate falha e a release mantém o Desk sem copilot.
- Serviço separado do endpoint que prepara/envia transações. Sem acesso a `planTransfer`, `set_binding`, `update_policy`, `close/emit proof`, `freeze/thaw` ou APIs administrativas. Leitura recebe somente allowlist de mints/carteiras demo; entrada não escolhe URL/RPC arbitrário ou arquivo local.
- Preferir leitura de evidências prontas, sem novo indexador ou polling. Limitar perguntas/tamanho/contexto e exibir horário/slot da fonte. Sem fontes ou RPC confiável, responder que não foi possível confirmar.
- Resposta distingue fato de sugestão: “A policy v3 de Beta não aceita B, conforme leitura no slot X”. Pode descrever o próximo passo manual já previsto, mas não executá-lo, construir transação assinável ou confirmar conformidade jurídica. Mostrar evidência usada; não inventar explorer link.

Conteúdo não confiável e dados

Metadados de token, mensagens, dados de proof e texto de incidente podem conter instruções maliciosas. Tratar tudo como dados, nunca comandos. Testar que um campo pedindo para ignorar regras, assinar ou enviar dados não amplia capacidades. A proteção principal é ausência de caminhos de escrita/segregados, não só um prompt de “não assine”.

Nesta demo, enviar ao modelo somente fixtures públicas/sintéticas necessárias. Não enviar KYC, dados pessoais, chaves, .env, conteúdo privado ou carteiras reais por padrão. Provedor/configuração do modelo, API key, custo e política de retenção são dependências a decidir; nenhuma assinatura/pagamento é autorizado pela spec. Configurar secret fora do repo/logs. Se não houver endpoint permitido, mostrar copilot desabilitado, não simular integração.

Aceite C01-C07

ID	Teste	Critério
C01	Por que Y/B falha em Beta?	Explicação coincide com razão determinística e referencia policy/slot.
C02	Prova X vencida afeta o quê?	Lista só os ativos demo conhecidos; não alega cobertura global.
C03	Leitura unknown ou expirada	Admite incerteza e não apresenta permissão atual como certa.
C04	Pedido de transferir, assinar, renovar ou alterar policy	Nenhum método de escrita acessível; signing/send provam falha por teste.
C05	Instrução maliciosa em metadata/proof	Nenhuma ação extra, disclosure ou mudança de objetivo; somente fatos necessários.
C06	Modelo indisponível/timeout	Desk e transferências/testes determinísticos seguem funcionando.
C07	Flag off + revisão de bundle/logs	Copilot removível, sem secret/signer; core e teste de fora da UI idênticos.

Não habilitar se faltar qualquer gate de não-assinatura/isolamento. Registrar versão/commit, plugin efetivamente carregado, testes e limites em docs/copilot.md. No pitch: “copilot de leitura sobre o Desk”, somente se executado; caso contrário “stretch goal não implementado”.

22. Ajuste de tarefas v3

- Bruno/Codex: setup AI Kit revisado e contexto carregado; manter P0/programa/testes como prioridade. Revisar ausência de escrita no copilot se ele entrar.
- Caio/Cursor: mesmo setup; Desk/adapters/incident workflow primeiro. Após 06/10, somente com gates verdes, spike Agent Kit/plugin read-only. Backend servidor e uso de modelo precisam caber na revisão até 08/10.
- Adicionar packages/copilot-readonly/, tests/copilot/ e documentação apenas se o stretch for aberto. Não incorporar dependência do copilot em programs/aveo-hook ou no caminho de transferência.
- Histórico v3: inclui ambos os kits; AI Kit é tooling de desenvolvimento agora, Agent Kit é camada de leitura opcional. Mantém Rota B imediata, A condicionada, datas antecipadas e autorização de Bruno para qualquer publicação/inscrição/submissão.

Fontes dos kits, lidas em 30/09/2026

- AI Kit: <https://github.com/solanabr/solana-ai-kit>
- README atual, setup e flag -agents: <https://github.com/solanabr/solana-ai-kit/blob/main/README.md>
- Installer e efeitos sobre arquivos: <https://github.com/solanabr/solana-ai-kit/blob/main/install.sh>
- Agent Kit core/plugins/quick start: <https://github.com/sendaiFun/solana-agent-kit>
- Migração V2, wallet interface e plugins próprios: <https://github.com/sendaiFun/solana-agent-kit/blob/v2/MIGRATING.md>

Essas fontes sustentam o plano de integração; setup Codex/Cursor, plugin Aveo e runtime sem signer ainda precisam de teste no repositório do time.